SafeDiffuser: Safe Planning with Diffusion Probabilistic Models

Wei Xiao*, **Tsun-Hsuan Wang, Chuang Gan, Daniela Rus**

Massachusetts Institute of Technology (MIT)

arXiv:2306.00148v1 [cs.LG] 31 May 2023

Diffusion model-based approaches have shown promise in data-driven planning, but there are no safety guarantees, thus making it hard to be applied for safety-critical applications. To address these challenges, we propose a new method, called SafeDiffuser, to ensure diffusion probabilistic models satisfy specifications by using a class of control barrier functions. The key idea of our approach is to embed the proposed finite-time diffusion invariance into the denoising diffusion procedure, which enables trustworthy diffusion data generation. Moreover, we demonstrate that our finite-time diffusion invariance method through generative models not only maintains generalization performance but also creates robustness in safe data generation. We test our method on a series of safe planning tasks, including maze path generation, legged robot locomotion, and 3D space manipulation, with results showing the advantages of robustness and guarantees over vanilla diffusion models†.

1. Introduction

Data-driven approaches have received increasing attentions due to their representation flexibility. Diffusion models Sohl-Dickstein et al. (2015) Ho et al. (2020) are data-driven generative models whose primary applications are in image generations Dhariwal and Nichol (2021) Du et al. (2020b) Saharia et al. (2022). Recently, diffusion models, termed diffusers Janner et al. (2022), have shown promise in trajectory planning for a variety of robotic tasks. Diffusers enable flexible behavior synthesis that makes it well generalized in novel environments.

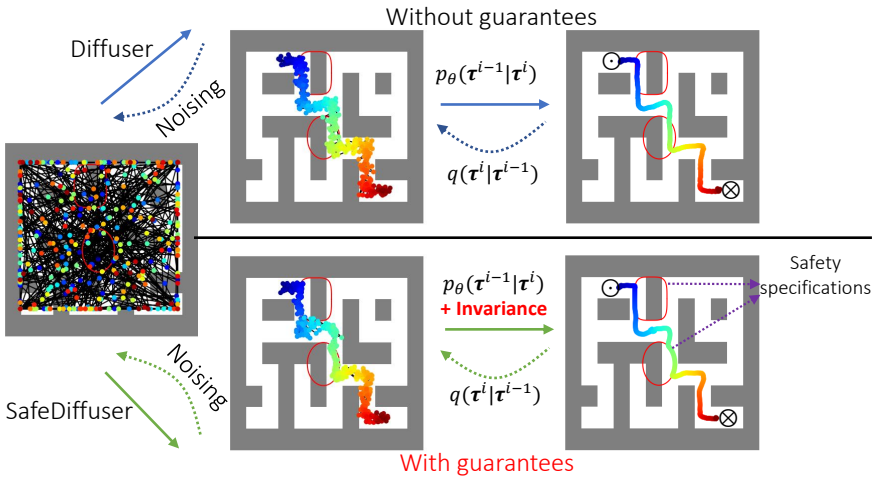

Fig. 1: Our proposed SafeDiffuser (lower) generates safe trajectories with guarantees, while the diffuser (upper) fails (from $\odot$ to $\otimes$).

During inference, the diffuser, conditioned on the current state and objectives, starts from Gaussian noise to generate clean planning trajectories based on which we can get a control policy. After applying this control policy one step forward, we get a new state and run the diffusion procedure again to get a new planning trajectory. This process is repeated until the objective is achieved. However, one big challenge in this method is that there are no safety guarantees. For instance, the planning trajectory could easily violate safety constraints in the maze (as shown in Fig. **1**). This shortcoming demands a fundamental fix to diffusion models to ensure the safe generation of planning trajectories in safety-critical applications such as trustworthy policy learning and optimization.

*Correspondence E-mail: weixy@mit.edu

†Videos can be found in the anonymous website: https://safediffuser.github.io/safediffuser/

In this paper, we propose to ensure diffusion models with specification guarantees using finite-time diffusion invariance. An invariance set is a form of specification mainly consisting of safety constraints in planning tasks. We ensure that diffusion models are invariant to uncertainties in diffusion procedure. We achieve safety by combining receding horizon control with stable diffusion. In receding horizon control, we compute safe paths incrementally. The key insight is to replace each path computation with diffusion-based path generation, allowing a broader exploration of the path space and makes it relatively easy to include additional constraints. The computed path is combined with simulation to validate that it can be safely actuated.

To ensure diffusers with specifications guarantees, we first find diffusion dynamics for the denoising diffusion procedure. Then, we use Lyapunov-based methods with forward invariance properties such as a class of control barrier functions (CBFs) Ames et al. (2017) Glotfelter et al. (2017) Nguyen and Sreenath (2016) Xiao and Belta (2019), to formally guarantee the satisfaction of specifications at the end of the diffusion procedure. CBFs works well in planning time using robot dynamics. However, doing this in diffusion models poses extra challenges since the generated data is not directly associated with robot dynamics which makes the use of CBFs non-trivial. As oppose to existing literature, 1. we propose to embed invariance into the diffusion time for diffusers. Thus, finite-time invariance is required in diffusers since specifications are usually violated as the trajectory is initially Gaussian noise. 2. We propose to add diffusion time components in invariance to address local trap problems that are prominent in planning. 3. We propose a quadratic program approach to incorporate finite-time diffusion invariance into the diffusion to maximally preserve the performance.

In summary, we make the following **new contributions**:

- We propose formal guarantees for diffusion probabilistic models via control-theoretic invariance.
- We propose a novel notion of finite-time diffusion invariance, and use a class of CBFs to incorporate it into the diffusion time of the procedure. We proposed three different safe diffusers, and show how we may address the local trap problem from specifications that are prominent in planning tasks.
- We demonstrate the effectiveness of our method on a variety of planning tasks using diffusion models, including safe planning in maze, robot locomotion and manipulation.

2. Preliminaries

In this section, we provide background on diffusion models and forward invariance in control theory.

Diffusion Probabilistic Models. Diffusion probabilistic models Sohl-Dickstein et al. (2015) Ho et al. (2020) Janner et al. (2022) are latent variable models representing a data generation process as an iterative denoising procedure $p_\theta(\boldsymbol{\tau}^{i-1}|\boldsymbol{\tau}^i)$, $i \in \{1,\dots,N\}$, where $\boldsymbol{\tau}^1,\dots,\boldsymbol{\tau}^N$ are latent variables of the same dimensionality of the clean (noiseless) data $\boldsymbol{\tau}^0 \sim q(\boldsymbol{\tau}^0)$, and N is the total denoising steps. This denoising procedure is the reverse of a forward diffusion process $q(\boldsymbol{\tau}^i|\boldsymbol{\tau}^{i-1})$ that gradually corrupts the clean data by adding noise. The denoising data generation is denoted by

$$p_\theta(\boldsymbol{\tau}^0) = \int p_\theta(\boldsymbol{\tau}^{0:N}) d\boldsymbol{\tau}^{1:N} = \int p(\boldsymbol{\tau}^N) \prod_{i=1}^{N} p_\theta(\boldsymbol{\tau}^{i-1}|\boldsymbol{\tau}^i) d\boldsymbol{\tau}^{1:N}, \tag{1}$$

where $p(\boldsymbol{\tau}^N)$ is a standard Gaussian prior distribution, and the joint distribution $p_\theta(\boldsymbol{\tau}^{0:N})$ is defined as a Markov chain with learned Gaussian transitions starting at $p(\boldsymbol{\tau}^N)$. The parameter θ is optimized by minimizing the usual variational bound on the negative log-likelihood of the reverse process: $\theta^* = \arg\min_\theta \mathbb{E}_{\boldsymbol{\tau}^0}\left[-\mathbf{log}\, p_\theta(\boldsymbol{\tau}^0)\right]$. The forward diffusion process $q(\boldsymbol{\tau}^i|\boldsymbol{\tau}^{i-1})$ is usually prespecified. The reverse process is often parameterized as Gaussian with time-dependent mean and variance.

Notations. There are two "times" involved in the paper: that of the diffusion process and that of the planning horizon. We use superscripts (i when unspecified) to denote the diffusion time of a trajectory (state) and subscripts (k when unspecified) to denote the planning time of a state on the trajectory. For instance, $\boldsymbol{\tau}^0$ denotes the planning trajectory at denoising diffusion time step 0 (i.e., a noiseless trajectory), and $\boldsymbol{x}_k^0$ denotes the state on the trajectory at planning time step k during the denoising diffusion time step 0 (i.e., a noiseless state). We note it as $\boldsymbol{x}_k = \boldsymbol{x}_k^0$ ($\boldsymbol{\tau} = \boldsymbol{\tau}^0$ as well) when there is no ambiguity. Further, a trajectory $\boldsymbol{\tau}^i$ is defined as a planning-time sequence of discretized states, i.e., $\boldsymbol{\tau}^i = (\boldsymbol{x}_0^i, \boldsymbol{x}_1^i, \ldots, \boldsymbol{x}_k^i, \ldots, \boldsymbol{x}_H^i)$, where $H \in \mathbb{N}$ is the planning horizon. During the denoising diffusion procedure, the diffusion time changes from N to 0, while the planning time varies from 0 to H.

Forward Invariance in Control Theory. Consider an affine control system of the form:

$$\dot{\boldsymbol{x}}_t = f(\boldsymbol{x}_t) + g(\boldsymbol{x}_t)\boldsymbol{u}_t \tag{2}$$

where $\boldsymbol{x}_t \in \mathbb{R}^n$, $f : \mathbb{R}^n \to \mathbb{R}^n$ and $g : \mathbb{R}^n \to \mathbb{R}^{n\times q}$ are locally Lipschitz, and $\boldsymbol{u}_t \in U \subset \mathbb{R}^q$, where U denotes a control constraint set. $\dot{\boldsymbol{x}}_t$ denotes the (planning) time derivative of state $\boldsymbol{x}_t$.

Definition 1. ***(Set invariance)****: A set $C \subset \mathbb{R}^n$ is forward invariant for system (2) if its solutions for some $\boldsymbol{u} \in U$ starting at any $\boldsymbol{x}_0 \in C$ satisfy $\boldsymbol{x}_t \in C$, $\forall t \geq 0$.*

Definition 2. ***(Extended class $\mathscr{K}$ function*** *Khalil (2002))****:*** *A Lipschitz continuous function $\alpha : [-b, a) \to (-\infty, \infty), b > 0, a > 0$ belongs to extended class $\mathscr{K}$ if it is strictly increasing and $\alpha(0) = 0$.*

Consider a safety constraint $b(\boldsymbol{x}_t) \geq 0$ for system (2), where $b : \mathbb{R}^n \to \mathbb{R}$ is continuously differentiable, we define a safe set in the form: $C := \{\boldsymbol{x}_t \in \mathbb{R}^n : b(\boldsymbol{x}_t) \geq 0\}$.

Definition 3. ***(Control Barrier Function (CBF)*** *Ames et al. (2017))****:*** *A function $b : \mathbb{R}^n \to \mathbb{R}$ is a CBF if there exists an extended class $\mathscr{K}$ function α such that*

$$\sup_{\boldsymbol{u}_t \in U} [L_f b(\boldsymbol{x}_t) + [L_g b(\boldsymbol{x}_t)]\boldsymbol{u}_t + \alpha(b(\boldsymbol{x}_t))] \geq 0, \tag{3}$$

for all $\boldsymbol{x}_t \in C$. L_f and L_g denote Lie derivatives w.r.t. $\boldsymbol{x}$ along f and g, respectively.

Theorem 1 (Ames et al. (2017))**.** *Given a CBF $b(\boldsymbol{x}_t)$ from Def. 3, if $\boldsymbol{x}_0 \in C$, then any Lipschitz continuous controller $\boldsymbol{u}_t$ that satisfies the constraint in (3), $\forall t \geq 0$ renders C forward invariant for system (2).*

If we need to differentiate $b(\boldsymbol{x}_t)$ more than once along the dynamics (2) until the control $\boldsymbol{u}_t$ explicitly shows, we use a high-order CBF Nguyen and Sreenath (2016) Xiao and Belta (2019) as a general form of CBF to guarantee safety for (2). In this work, we map the forward invariance in control theory to finite time *diffusion invariance in diffusion models*, where we incorporate CBFs into the diffusion time as opposed to their regular applications in planning time. In addition, we show how we may address the local traps during diffusion.

3. Safe Diffuser

In this section, we propose three different safe diffusers to ensure the safe generation of data in diffusion, i.e., to ensure the satisfaction of specifications $b(\boldsymbol{x}_k) \geq 0, \forall k \in \{0, \ldots, H\}$. Each of the proposed safe diffusers have their own flexibility, such as avoiding local traps in planning. We consider discretized system states in the sequel. Safety in continuous planning time can be guaranteed using a lower hierarchical control framework employing other CBFs, as in Ames et al. (2017); Nguyen and Sreenath (2016); Xiao and Belta (2019).

In the denoising diffusion procedure, since the learned Gaussian transitions starts at $p(\boldsymbol{x}^N) \sim \mathcal{N}(0, \boldsymbol{I})$, it is highly likely that specifications are initially violated, i.e., $\exists k \in \{0,\dots,H\}, b(\boldsymbol{x}_k^N) < 0$. For safe data generation, we wish to have $b(\boldsymbol{x}_k^0) \geq 0$(*i.e.,* $b(\boldsymbol{x}_k) \geq 0), \forall k \in \{0,\dots,H\}$. Since the maximum denoising diffusion step N is limited, this needs to be guaranteed in a finite diffusion time step. Therefore, we propose the finite-time diffusion invariance of the diffusion procedure as follows:

Definition 4 (Finite-time Diffusion Invariance)**.** *If there exists $i \in \{0,\dots,N\}$ such that $b(\boldsymbol{x}_k^j) \geq 0, \forall k \in \{0,\dots,H\}, \forall j \leq i$, then a denoising diffusion procedure $p_\theta(\boldsymbol{\tau}^{i-1}|\boldsymbol{\tau}^i), i \in \{1,\dots,N\}$ with respect to a specification $b(\boldsymbol{x}_k) \geq 0, \forall k \in \{0,\dots,H\}$ is finite-time diffusion invariant.*

The above definition can be interpreted as that if $b(\boldsymbol{x}_k^N) \geq 0, k \in \{0,\dots,H\}$, then we require $b(\boldsymbol{x}_k^i) \geq 0, \forall i \in \{0,\dots,N\}$ (similar to the forward invariance definition as in Def. 1); otherwise, we require that $b(\boldsymbol{x}_k^j) \geq 0, \forall j \in \{0,\dots,i\}, i \in \{0,\dots,N\}$, where i is a finite diffusion time.

In the following, we propose three different methods to achieve finite-time diffusion invariance. The first method is a general form of the safe- diffuser, and the other two are variants to address local traps in planning.

3.1. Robust-Safe Diffuser

The safe denoising diffusion procedure is considered at every diffusion step. Following (1), the data generation at the diffusion time $j \in \{0,\dots,N-1\}$ is given by:

$$p_\theta(\boldsymbol{\tau}^j) = \int p(\boldsymbol{\tau}^N) \prod_{i=j+1}^{N} p_\theta(\boldsymbol{\tau}^{i-1}|\boldsymbol{\tau}^i) d\boldsymbol{\tau}^{j+1:N} \quad (4)$$

A sample $\boldsymbol{\tau}^j, j \in \{0,\dots,N-1\}$ follows the data distribution in (4), i.e., we have

$$\boldsymbol{\tau}^j \sim p_\theta(\boldsymbol{\tau}^j). \quad (5)$$

The denoising diffusion dynamics are then given by:

$$\dot{\boldsymbol{\tau}}^j = \lim_{\Delta\tau \to 0} \frac{\boldsymbol{\tau}^j - \boldsymbol{\tau}^{j+1}}{\Delta\tau} \quad (6)$$

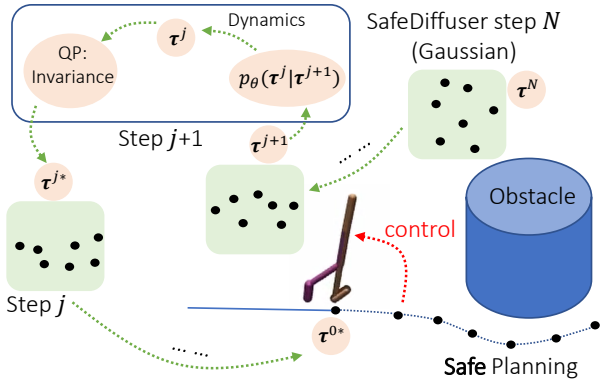

Fig. 2: The proposed SafeDiffuser workflow. SafeDiffuser performs an additional step of invariance QP solver in the diffusion dynamics to ensure safety. The final control signal is inferred from safe planning trajectories.

where $\dot{\boldsymbol{\tau}}$ is the (diffusion) time derivative of $\boldsymbol{\tau}$. $\Delta\tau > 0$ is a small enough diffusion time step length during implementations, and $\boldsymbol{\tau}^{j+1}$ is available from the last diffusion step.

In order to impose finite-time diffusion invariance on the diffusion procedure, we wish to make diffusion dynamics (6) controllable. We reformulate (6) as

$$\dot{\boldsymbol{\tau}}^j = \boldsymbol{u}^j, \quad (7)$$

where $\boldsymbol{u}^j$ is a control variable of the same dimensionality as $\boldsymbol{\tau}^j$. On the other hand, we wish $\boldsymbol{u}^j$ to stay close to $\frac{\boldsymbol{\tau}^j - \boldsymbol{\tau}^{j+1}}{\Delta\tau}$ in order to maximally preserve the performance of the diffusion model. The above model can be rewritten in terms of each state on the trajectory $\boldsymbol{\tau}^j$: $\dot{\boldsymbol{x}}_k^j = \boldsymbol{u}_k^j$, where $\boldsymbol{u}_k^j$ is the kth component of $\boldsymbol{u}^j$. Then, we can define CBFs to ensure the satisfaction of $b(\boldsymbol{x}_k^j) \geq 0$ (in finite diffusion time): $h(\boldsymbol{u}_k^j|\boldsymbol{x}_k^j) := \frac{db(\boldsymbol{x}_k^j)}{d\boldsymbol{x}_k^j}\boldsymbol{u}_k^j + \alpha(b(\boldsymbol{x}_k^j)) \geq 0, k \in \{0,\dots,H\}, j \in \{0,\dots,N-1\}$, where $\alpha(\cdot)$ is an extended class $\mathscr{K}$ function. We have the following theorem to show the finite-time diffusion invariance (proof is given in appendix):

Theorem 2. *Let the diffusion dynamics defined as in (6) whose controllable form is defined as in (7). If there exists an extended class $\mathscr{K}$ function α such that*

$$h(\boldsymbol{u}_k^j|\boldsymbol{x}_k^j) \geq 0, \forall k \in \{0,\ldots,H\}, \forall j \in \{0,\ldots,N-1\}, \tag{8}$$

where $h(\boldsymbol{u}_k^j|\boldsymbol{x}_k^j) = \frac{db(\boldsymbol{x}_k^j)}{d\boldsymbol{x}_k^j}\boldsymbol{u}_k^j + \alpha(b(\boldsymbol{x}_k^j))$, then the diffusion procedure $p_\theta(\boldsymbol{\tau}^{i-1}|\boldsymbol{\tau}^i), i \in \{1,\ldots,N\}$ is finite-time diffusion invariant with almost probability 1.

One possible issue in the robust-safe diffusion procedure is that if $b(\boldsymbol{x}_k^j) \geq 0$ when j is close to the initial diffusion step N, then the state $\boldsymbol{x}_k^j$ can never violate the specification after diffusion step $j < N$. When there are local traps from specifications, the state $\boldsymbol{x}_k^j$ may get stuck there during the denoising diffusion process, which may adversely affect the performance. In order to address this issue, we propose relaxed-safe diffuser and time-varying-safe diffuser in the following subsections.

3.2. Relaxed-Safe Diffuser

In order to address the local trap problems imposed by specifications during the denoising diffusion procedure, we propose a variation of the robust-safe diffuser. We define the diffusion dynamics and their controllable form as in (6) - (7). The modified versions for CBFs are in the form:

$$h(\boldsymbol{u}_k^j, r_k^j|\boldsymbol{x}_k^j) := \frac{db(\boldsymbol{x}_k^j)}{d\boldsymbol{x}_k^j}\boldsymbol{u}_k^j + \alpha(b(\boldsymbol{x}_k^j)) - w_k(j)r_k^j \geq 0, k \in \{0,\ldots,H\}, j \in \{0,\ldots,N-1\}, \tag{9}$$

where $r_k^j \in \mathbb{R}$ is a relaxation variable that is to be determined (shown in the next section). $w_k(j) \geq 0$ is a diffusion time-varying weight on the relaxation variable such that it gradually decrease to 0 as $j \to 0$.

When $w_k(j)$ decreases to 0, the condition (9) becomes a hard constraint. One problem in such cases is that the diffusion performance may be adversely affected by such a hard constraint. In order to address this issue, we may run additional $N_a \in \mathbb{N}$ diffusion steps, while setting the diffusion time to 0 when $j < 0$ in the reverse process. The corresponding CBF conditions to (9) are to change the domain of j to $j \in \{-N_a,\ldots,N-1\}$.We also have the following theorem to show the finite-time diffusion invariance (proof is given in appendix):

Theorem 3. *Let the diffusion dynamics defined as in (6) whose controllable form is defined as in (7). If there exist an extended class $\mathscr{K}$ function α, a large enough extra diffusion step $N_a \in \mathbb{N}$, and a time-varying weight $w_k(j)$ where $w_k(j) = 0$ for all $j \leq 0$ such that*

$$h(\boldsymbol{u}_k^j, r_k^j|\boldsymbol{x}_k^j) \geq 0, \forall k \in \{0,\ldots,H\}, \forall j \in \{-N_a,\ldots,N-1\}, \tag{10}$$

where $h(\boldsymbol{u}_k^j, r_k^j|\boldsymbol{x}_k^j) = \frac{db(\boldsymbol{x}_k^j)}{d\boldsymbol{x}_k^j}\boldsymbol{u}_k^j + \alpha(b(\boldsymbol{x}_k^j)) - w_k(j)r_k^j$, then the diffusion procedure $p_\theta(\boldsymbol{\tau}^{i-1}|\boldsymbol{\tau}^i), i \in \{-N_a,\ldots,N\}$ is finite-time diffusion invariant with almost probability 1.

After the denoising diffusion procedure is done at step $-N_a$, we would set the $\boldsymbol{\tau}^{-N_a}$ as the output data of the diffusion model, i.e., $\boldsymbol{\tau}^0 = \boldsymbol{\tau}^{-N_a}$.

3.3. Time-Varying-Safe Diffuser

As an alternative to the relaxed-safe diffuser, we propose another safe diffuser called time-varying-safe diffuser in this subsection. The proposed time-varying-safe diffuser can also address the local trap issues induced by specifications.

In this case, we directly modify the specification $b(\boldsymbol{x}_k^j) \geq 0$ by a diffusion time-varying function $\gamma_k : j \to \mathbb{R}$ in the form:

$$b(\boldsymbol{x}_k^j) - \gamma_k(j) \geq 0, k \in \{0,\dots,H\}, j \in \{0,\dots,N\}, \tag{11}$$

where $\gamma_k(j)$ is continuously differentiable, and is defined such that $\gamma_k(N) \leq b(\boldsymbol{x}_k^N)$ and $\gamma_k(0) = 0$.

The modified time-varying specification can then be enforced using CBFs: $h(\boldsymbol{u}_k^j|\boldsymbol{x}_k^j, \gamma_k(j)) := \frac{db(\boldsymbol{x}_k^j)}{d\boldsymbol{x}_k^j}\boldsymbol{u}_k^j - \dot{\gamma}_k(j) + \alpha(b(\boldsymbol{x}_k^j) - \gamma_k(j)) \geq 0, k \in \{0,\dots,H\}, j \in \{0,\dots,N-1\}$, where $\dot{\gamma}_k(j)$ is the diffusion time derivative of $\gamma_k(j)$. Finally, we have the following theorem to show the finite-time diffusion invariance (proof is given in appendix):

Theorem 4. *Let the diffusion dynamics defined as in (6) whose controllable form is defined as in (7). If there exist an extended class $\mathscr{K}$ function α and a time-varying function $\gamma_k(j)$ where $\gamma_k(N) \leq b(\boldsymbol{x}_k^N)$ and $\gamma_k(0) = 0$ such that*

$$h(\boldsymbol{u}_k^j|\boldsymbol{x}_k^j, \gamma_k(j)) \geq 0, \forall k \in \{0,\dots,H\}, \forall j \in \{0,\dots,N-1\}, \tag{12}$$

where $h(\boldsymbol{u}_k^j|\boldsymbol{x}_k^j, \gamma_k(j)) = \frac{db(\boldsymbol{x}_k^j)}{d\boldsymbol{x}_k^j}\boldsymbol{u}_k^j - \dot{\gamma}_k(j) + \alpha(b(\boldsymbol{x}_k^j) - \gamma_k(j))$, then the diffusion procedure $p_\theta(\boldsymbol{\tau}^{i-1}|\boldsymbol{\tau}^i), i \in \{0,\dots,N\}$ is finite-time diffusion invariant.

4. Enforcing Invariance in Diffuser

In this section, we show how we may incorporate the three proposed invariance methods from the last section into diffusion models. Enforcing the finite-time invariance in diffusion models is equivalent to ensure the satisfaction of the conditions in Thms. 2-4 in the diffusion procedure. In this section, we propose a minimum-deviation quadratic program (QP) approach to achieve that. We wish to enforce these conditions at every step of the diffusion (as shown in Fig. **2**) as those states that are far from the specification boundaries $b(\boldsymbol{x}_k^j) = 0$ can also be optimized accordingly, and thus, the model may generate more coherent trajectories.

Enforcing Invariance for Robust-Safe (RoS) and Time-Varying-Safe Diffusers. During implementation, the diffusion time step length $\Delta\boldsymbol{\tau}$ in (6) is chosen to be small enough, and we wish the control $\boldsymbol{u}^j$ to stay close to the right-hand side of (6). Thus, we can formulate the following QP-based optimization to find the optimal control for $\boldsymbol{u}^j$ that satisfies the condition in Thms. 2 or 4:

$$\boldsymbol{u}^{j*} = \arg\min_{\boldsymbol{u}^j} ||\boldsymbol{u}^j - \frac{\boldsymbol{\tau}^j - \boldsymbol{\tau}^{j+1}}{\Delta\tau}||^2, \text{ s.t., } (8) \text{ if RoS diffuser else s.t., } (12), \tag{13}$$

where $||\cdot||$ denotes the 2-norm of a vector. If we have more than one specification, we can add the corresponding conditions in Thm. 2 for each of them to the above QP. After we solve the above QP and get $\boldsymbol{u}^{j*}$, we update (7) by setting $\boldsymbol{u}^j = \boldsymbol{u}^{j*}$ within the time step and get a new state for the diffusion procedure. Note that all of these happen at the end of each diffusion step.

Enforcing Invariance for Relaxed-Safe Diffuser. In this case, since we have relaxation variables for each of the safety specification, we wish to minimize these relaxations in the cost function to drive all the state towards the satisfaction of specifications. In other words, we have the following QP:

$$\boldsymbol{u}^{j*}, r^{j*} = \arg\min_{\boldsymbol{u}^j, r^j} ||\boldsymbol{u}^j - \frac{\boldsymbol{\tau}^j - \boldsymbol{\tau}^{j+1}}{\Delta\tau}||^2 + ||r^j||^2, \text{ s.t., } (10), \tag{14}$$

where r^j is the concatenation of r_k^j for all $k \in \{0,\dots,H\}$. As an alternative, all the constraints above may share the same relaxation variable, i.e., the dimension of r^j is only one. After we solve the above QP and get $\boldsymbol{u}^{j*}$, we update (7) by setting $\boldsymbol{u}^j = \boldsymbol{u}^{j*}$ within the time step and get a new state.

Algorithm 1 Enforcing invariance in diffusion models

Input: the last trajectory of diffusion $\boldsymbol{\tau}^{j+1}$ at diffusion step $j \in \{0, \ldots, N\}$
Output: safe diffusion state $\boldsymbol{\tau}^{j*}$.
(a) Run diffusion procedure (4) and sample (5) as usual at step j and get $\boldsymbol{\tau}^{j}$.
(b) Find diffusion dynamics as in (6) - (7).
if *Robust-safe diffuser* **then**
  Formulate the QP (13), solve it and get $\boldsymbol{u}^{j*}$.
else if Relaxed-safe diffuser **then**
  Define the time-varying weight $w_k(j)$ in (9), formulate the QP (14), solve it and get $\boldsymbol{u}^{j*}, r^{j*}$.
else
  Design the time-varying function $\gamma_k(j)$ in (11), formulate the QP (13), solve it and get $\boldsymbol{u}^{j*}$.
end if
(c) Update dynamics (7) with $\boldsymbol{u}^{j} = \boldsymbol{u}^{j*}$ and get $\boldsymbol{\tau}^{j*}$. Finally, $\boldsymbol{\tau}^{j} \leftarrow \boldsymbol{\tau}^{j*}$.

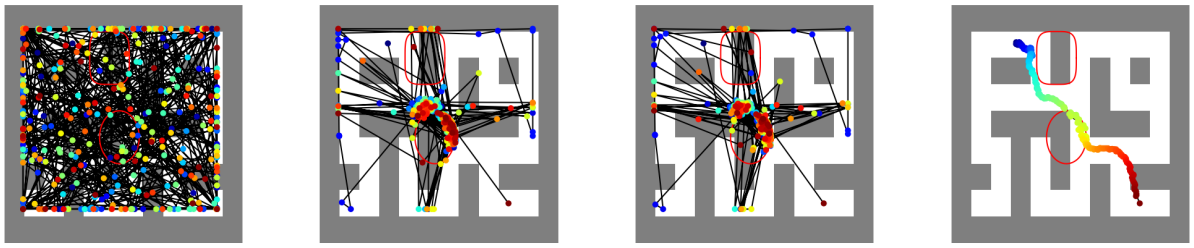

Fig. 3: Maze planning (blue to red) denoising diffusion procedure with classifier-based guidance (Left to right: diffusion time steps 256, 4, 3, -50, respectively). Red ellipse and super-ellipse (outside) denote safe specifications. The safe classifier-based guidance approach adversely affects the diffusion procedure without guarantees.

Complexity of enforcing invariance The computational complexity of a QP is $\mathcal{O}(q^3)$, where q is the dimension of the decision variable. When there is a set S of specifications, we just add the corresponding constraints for each specification the QP. The complexity of the three proposed safe diffuser are similar.

The algorithm for enforcing invariance is straight forward, which includes the construction of proper conditions, the solving of QP, and the update of diffusion state. We summary the algorithm in Alg. 1.

5. Experiments

We set up experiments to answer the following questions:
- Does our method match the theoretical potential in various tasks quantitatively and qualitatively?
- How does our method compare with state-of-the-art approaches in enforcing safety specifications?
- How does our proposed method affect the performance of diffusion under guaranteed specifications?

5.1. Safe Planning in Maze

In this experiment, we aim to impose trajectory constraints on the planning path of a maze. The training data is publicly available from Janner et al. (2022), in which initial positions and destinations in maze are randomly generated. The diffusion model is conditioned on the initial positions and destinations.

Table 1: Maze safe planning comparisons with benchmarks. Items are short for satisfaction of simple specifications (S-SPEC) and complex specifications (C-SPEC), score of planning tasks (SCORE), computation time at each diffusion step (TIME) in seconds, respectively. In the method column, items are short for robust-safe diffuser (RoS-DIFFUSER), relaxed-safe diffuser (ReS-DIFFUSER), and time-varying-safe diffuser (TVS-DIFFUSER), respectively. The classifier guidance$-\varepsilon$ method applies (safe) gradient to the model when the state is $\varepsilon > 0$ close to the boundary.

METHOD	S-SPEC(↑ & ≥ 0)	C-SPEC(↑ & ≥ 0)	SCORE (↑)	TIME
DIFFUSER (BASELINE) JANNER ET AL. (2022)	-0.983	-0.894	1.016±0.712	0.007
TRUNCATE BROCKMAN ET AL. (2016)	$-1.192e^{-7}$	-0.759	0.754±0.779	0.024
CLASSIFIER GUIDANCE DHARIWAL AND NICHOL (2021)	-0.789	-0.979	0.502±0.328	0.053
CLASSIFIER GUIDANCE$-\varepsilon$ DHARIWAL AND NICHOL (2021)	-0.853	-0.995	0.470±0.366	0.061
ROS-DIFFUSER (OURS)	$-2.384e^{-7}$	$-5.960e^{-7}$	0.770±0.782	0.106
RES-DIFFUSER (OURS)	$-2.384e^{-7}$	$-4.768e^{-7}$	0.762±0.746	0.107
TVS-DIFFUSER (OURS)	$-2.384e^{-7}$	$-5.364e^{-7}$	0.806±0.783	0.107

The diffuser cannot guarantee the satisfaction of any specifications, as shown in Fig. **1**. When using classifier-based guidance in diffusion for safety specifications, the performance could be significantly affected (Fig. **3**). As a result, the generated trajectory will largely deviates from the desired one with no safety. The proposed robust-safe diffuser (RoS-diffuser), relaxed-safe diffuser (ReS-diffuser), and time-varying-safe diffuser (TVS-diffuser) can all guarantee the satisfaction of specifications, even when the specifications are complex (as long as they are differentiable), as shown in Table **1**. The satisfaction scores of the proposed methods are not strictly positive, and this may be due to the computation errors or inter-sampling effect as even the truncation method cannot strictly satisfy the simple specifications. The proposed methods can also maximally preserve the performance of diffusion models, as shown by the score in Table **1**, as well as in Fig. **4**. The ReS-diffuser and TVS-diffuser can address the local trap problem from specifications, as shown by figures in appendix.

5.2. Safe Planning for Robot Locomotion

For robot locomotion (in MuJoCo), we wish the robot to avoid collisions with obstacles, such as the roof. In this case, since there is no local trap problem, we only consider robust-safe diffuser (RoS-diffuser). Others work similarly. The training data set is publicly available from Janner et al. (2022).

As expected, collisions with roof are very likely to happen in walker and hopper using the diffuser since there are no guarantees, as shown in Table **2**. The truncation method can work for simple specifications (S-spec), but not for complex specifications (C-spec). The classifier-based guidance can improve the satisfaction of specifications, but with no guarantees. Collision-free is guaranteed using the proposed Ros-diffuser, and one example of diffusion procedure is shown in Fig. **5**.

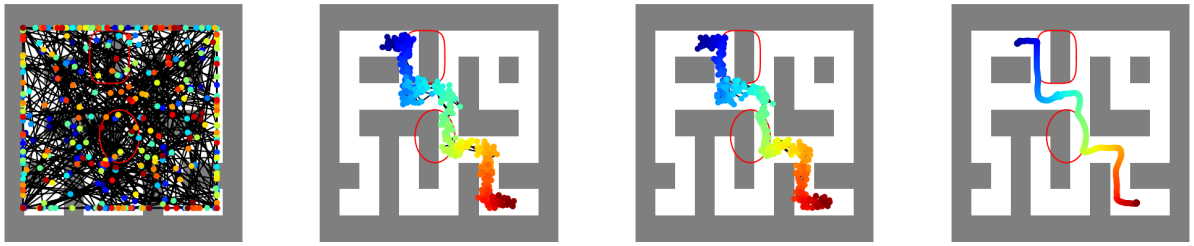

Fig. 4: Maze planning (blue to red) denoising diffusion procedure with the proposed time-varying safe diffuser (Left to right: diffusion time steps 256, 4, 3, -50, respectively). Red ellipse and super-ellipse (outside) denote safe specifications. The proposed time-varying safe diffuser can guarantee specifications at the end of diffusion while not significantly affecting the diffusion procedure.

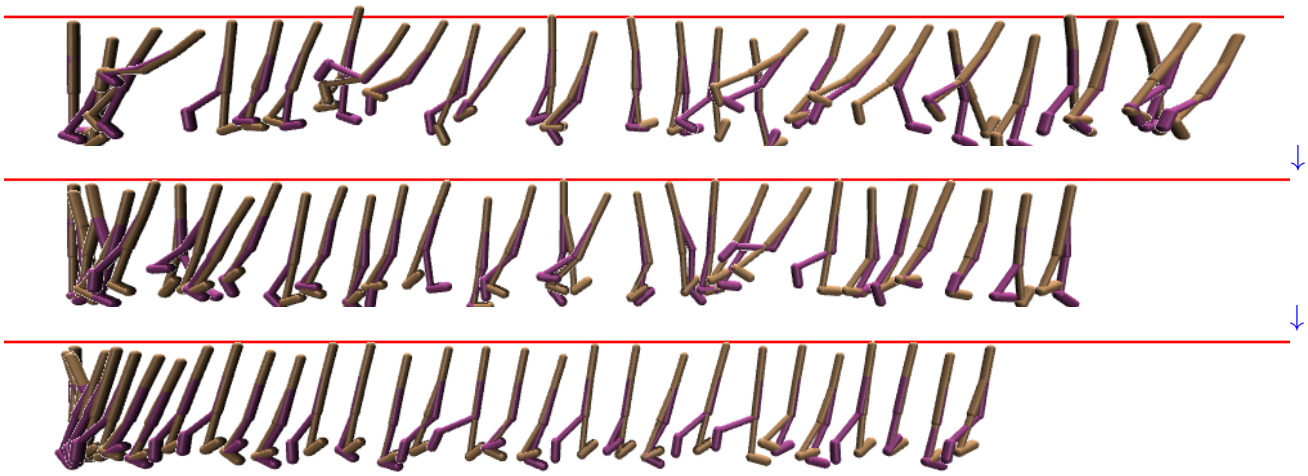

Fig. 5: Walker2D planning denoising diffusion procedure with the proposed robust-safe diffuser (Up to down: diffusion time steps 20, 10, 0, respectively). The red line denotes the roof the walker needs to safely avoid during locomotion (safety specifications). Safety is violated at step 20 since the trajectory is initially Gaussian noise, but is eventually guaranteed (step 0).

5.3. Safe Planning for Manipulation

For manipulation (in Pybullet), the diffusion models generate joint trajectories (as controls) for the robot, which are conditioned on the locations of the objects to grasp and place. The training data set is publicly available from Janner et al. (2022). Specifications are joint limitations to avoid collision in joint space.

In this case, the truncation method still fails to work for complex specifications (speed-dependent joint limitations). Our proposed RoS-diffuser can work for all specifications as long as they are differentiable. An interesting observation is that the proposed RoS-diffuser can even improve the performance (reward) of diffusion models in this case, as shown in Table **3**. This may be due to the fact that the satisfaction of joint limitations can avoid collision in the joint space of the robot as Pybullet is a physical simulator. The computation time of the proposed RoS-diffuser is comparable to other methods. An illustration for the safe diffusion and manipulation procedure is shown in Fig. **6**.

Table 2: Robot safe planning comparisons with benchmarks. Items are short for satisfaction of simple specifications (S-SPEC), satisfaction of complex specifications (C-SPEC), score of planning tasks (SCORE), computation time at each diffusion step (TIME) in seconds, respectively.

EXPERIMENT	METHOD	S-SPEC(↑ & ≥ 0)	C-SPEC(↑ & ≥ 0)	SCORE (↑)	TIME
WALKER2D	DIFFUSER (BASELINE) JANNER ET AL. (2022)	- 9.375	-4.891	0.346±0.106	0.037
	TRUNCATE BROCKMAN ET AL. (2016)	0.0	×	0.286±0.180	0.105
	CLASSIFIER GUIDANCE DHARIWAL AND NICHOL (2021)	- 0.575	-0.326	0.208±0.140	0.053
	ROS-DIFFUSER (OURS)	0.0	$-6.706e^{-8}$	0.312±0.782	0.183
HOPPER	DIFFUSER (BASELINE) JANNER ET AL. (2022)	- 2.180	-1.862	0.455±0.038	0.038
	TRUNCATE BROCKMAN ET AL. (2016)	0.0	×	0.436±0.067	0.046
	CLASSIFIER GUIDANCE DHARIWAL AND NICHOL (2021)	- 0.894	-0.524	0.478±0.038	0.047
	ROS-DIFFUSER (OURS)	0.0	$-5.960e^{-8}$	0.430±0.040	0.170

6. Related Works

Diffusion models and planning Diffusion models Sohl-Dickstein et al. (2015) Ho et al. (2020) are data-driven generative modeling tools, widely used in applications to image generations Dhariwal and Nichol (2021) Du et al. (2020b), in planning Hafner et al. (2019) Janner et al. (2021) Ozair et al. (2021) Janner et al. (2022), and in language Saharia et al. (2022) Liu et al. (2023). Generative models are combined with reinforcement learning to explore dynamic models in the form of convolutional U-networks Kaiser et al. (2019), stochastic recurrent networks Ke et al. (2019), neural ODEs Du et al. (2020a), generative adversarial networks Eysenbach et al. (2022), neural radiance fields Li et al. (2022), and transformers Chen et al. (2022). Further, planning tasks are becoming increasingly important for diffusion models Lambert et al. (2021) Ozair et al. (2021) Janner et al. (2022) as they can generalize well in all kinds of robotic problems. However, there are no methods to equip diffusion models with specification guarantees, which is especially important for safety-critical applications. Here, we address this issue using the proposed finite-time diffusion invariance.

Set invariance and CBFs. An invariant set has been widely used to represent the safe behavior of dynamical systems Preindl (2016) Rakovic et al. (2005) Ames et al. (2017) Glotfelter et al. (2017) Xiao et al. (2023a). In the state of the art of control, Control Barrier Functions (CBFs) are also widely used to prove set invariance Aubin (2009), Prajna et al. (2007), Wisniewski and Sloth (2013). CBFs can be traced back to optimization problems Boyd and Vandenberghe (2004), and are Lyapunov-like functions Wieland and Allgöwer (2007). For time-varying systems, CBFs can also be adapted accordingly Lindemann and Dimarogonas (2018). Existing CBF approaches are usually applied in planning time since they are closely coupled with system dynamics. There are few studies of CBFs in other space, such as the diffusion time horizon in diffusion models. Our work

Table 3: Manipulation safe planning comparisons with benchmarks. Items are short for satisfaction of simple specifications (S-SPEC), satisfaction of complex specifications (C-SPEC), reward of planning tasks (REWARD), computation time at each diffusion step (TIME) in seconds, respectively. In the method column, the item is short for robust-safe diffuser (RoS-DIFFUSER).

METHOD	S-SPEC(↑ & ≥ 0)	C-SPEC(↑ & ≥ 0)	REWARD (↑)	TIME
DIFFUSER (BASELINE) JANNER ET AL. (2022)	-0.057	-0.065	0.650±0.107	0.038
TRUNCATE BROCKMAN ET AL. (2016)	$1.631e^{-8}$	×	0.575±0.112	0.069
CLASSIFIER GUIDANCE DHARIWAL AND NICHOL (2021)	-0.050	-0.053	0.800±0.328	0.075
RoS-DIFFUSER (OURS)	0.072	0.069	0.925±0.107	0.088

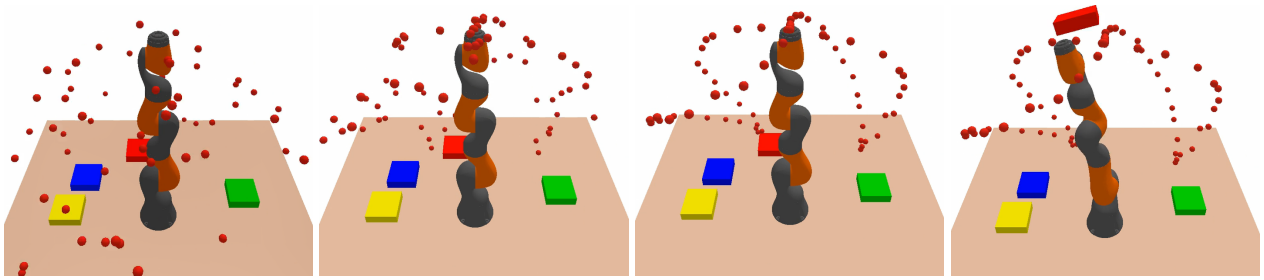

Fig. 6: Manipulation planning denoising diffusion procedure with the proposed robust-safe diffuser (Left to right: diffusion time steps 1000, 100, 0, and execution time step 100, respectively). The red dots denote the planning trajectory of the end-effector.

addresses all these limitations.

Guarantees in neural networks. Differentiable optimization methods show promise for neural network controllers with guarantees Amos et al. (2018); Pereira et al. (2020); Xiao et al. (2023a). They are usually served as a layer (filter) in the neural networks. In Amos and Kolter (2017), a differentiable quadratic program (QP) layer, called OptNet, was introduced. OptNet with CBFs has been used in neural networks as a filter for safe controls Pereira et al. (2020), in which CBFs are not trainable, thus, potentially limiting the system's learning performance. In Deshmukh et al. (2019); Ferlez et al. (2020); Zhao et al. (2021), safety guaranteed neural network controllers have been learned through verification-in-the-loop training. The verification approaches cannot ensure coverage of the entire state space. More recently, CBFs have been incorporated into neural ODEs to equip them with specification guarantees Xiao et al. (2023b). However, none of these methods can be applied in diffusion models, which we address in this paper.

7. Conclusions, Discussions and Future Work

We have proposed finite-time diffusion invariance for diffusion models to ensure safe planning for safety-critical applications. We have demonstrated the effectiveness of our method on a series of robotic planning tasks. Nonetheless, our method face a few shortcomings motivating for future work.

Specifically, specifications for diffusion models should be expressed as continuously differentiable constraints that may be unknown for planning tasks. Further work may explore how to learn specifications from history trajectory data. There is also a gap between planning and control using diffusion models. We may further investigate diffusion for safe control policies when robot dynamics are known or to be learned.

Broader Impact. Our proposed finite-time diffusion invariance can be applied to guarantees in other tasks, such as image generations, in addition to planning. In such cases, we can ensure the generation of desired patterns/objects, such as a cat in the generated images. We will further investigate those directions in our future work.

8. Acknowledgement

The research was supported in part by Capgemini Engineering. It was also partially sponsored by the United States Air Force Research Laboratory and the United States Air Force Artificial Intelligence Accelerator and was accomplished under Cooperative Agreement Number FA8750-19-2-1000. The views and conclusions contained in this document are those of the authors and should not be interpreted as representing the official policies, either expressed or implied, of the United States Air Force or the U.S. Government. The U.S. Government is authorized to reproduce and distribute reprints for Government purposes notwithstanding any copyright notation herein. This research was also supported in part by the AI2050 program at Schmidt Futures (Grant G- 965 22-63172).

References

A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada. Control barrier function based quadratic programs for safety critical systems. *IEEE Transactions on Automatic Control*, 62(8):3861–3876, 2017.

B. Amos and J. Z. Kolter. Optnet: Differentiable optimization as a layer in neural networks. In *Proceedings of the 34th International Conference on Machine Learning - Volume 70*, pages 136–145, 2017.

B. Amos, I. D. J. Rodriguez, J. Sacks, B. Boots, and J. Z. Kolter. Differentiable mpc for end-to-end planning and control. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems*, page 8299–8310. Curran Associates Inc., 2018.

J.-P. Aubin. *Viability theory*. Springer, 2009.

S. P. Boyd and L. Vandenberghe. *Convex optimization*. Cambridge university press, New York, 2004.

G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba. Openai gym, 2016.

C. Chen, Y.-F. Wu, J. Yoon, and S. Ahn. Transdreamer: Reinforcement learning with transformer world models. *arXiv preprint arXiv:2202.09481*, 2022.

J. V. Deshmukh, J. P. Kapinski, T. Yamaguchi, and D. Prokhorov. Learning deep neural network controllers for dynamical systems with safety guarantees: Invited paper. In *2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pages 1–7, 2019.

P. Dhariwal and A. Nichol. Diffusion models beat gans on image synthesis. *Advances in Neural Information Processing Systems*, 34:8780–8794, 2021.

J. Du, J. Futoma, and F. Doshi-Velez. Model-based reinforcement learning for semi-markov decision processes with neural odes. *Advances in Neural Information Processing Systems*, 33:19805–19816, 2020a.

Y. Du, S. Li, and I. Mordatch. Compositional visual generation with energy based models. *Advances in Neural Information Processing Systems*, 33:6637–6647, 2020b.

B. Eysenbach, A. Khazatsky, S. Levine, and R. R. Salakhutdinov. Mismatched no more: Joint model-policy optimization for model-based rl. *Advances in Neural Information Processing Systems*, 35:23230–23243, 2022.

J. Ferlez, M. Elnaggar, Y. Shoukry, and C. Fleming. Shieldnn: A provably safe nn filter for unsafe nn controllers. *preprint arXiv:2006.09564*, 2020.

P. Glotfelter, J. Cortes, and M. Egerstedt. Nonsmooth barrier functions with applications to multi-robot systems. *IEEE control systems letters*, 1(2):310–315, 2017.

D. Hafner, T. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee, and J. Davidson. Learning latent dynamics for planning from pixels. In *International conference on machine learning*, pages 2555–2565. PMLR, 2019.

J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33:6840–6851, 2020.

M. Janner, Q. Li, and S. Levine. Offline reinforcement learning as one big sequence modeling problem. *Advances in neural information processing systems*, 34:1273–1286, 2021.

M. Janner, Y. Du, J. Tenenbaum, and S. Levine. Planning with diffusion for flexible behavior synthesis. In *International Conference on Machine Learning*, pages 9902–9915. PMLR, 2022.

L. Kaiser, M. Babaeizadeh, P. Milos, B. Osinski, R. H. Campbell, K. Czechowski, D. Erhan, C. Finn, P. Kozakowski, S. Levine, et al. Model-based reinforcement learning for atari. *arXiv preprint arXiv:1903.00374*, 2019.

N. R. Ke, A. Singh, A. Touati, A. Goyal, Y. Bengio, D. Parikh, and D. Batra. Modeling the long term future in model-based reinforcement learning. In *International Conference on Learning Representations*, 2019.

H. K. Khalil. *Nonlinear Systems*. Prentice Hall, third edition, 2002.

N. Lambert, A. Wilcox, H. Zhang, K. S. Pister, and R. Calandra. Learning accurate long-term dynamics for model-based reinforcement learning. In *2021 60th IEEE Conference on Decision and Control (CDC)*, pages 2880–2887. IEEE, 2021.

Y. Li, S. Li, V. Sitzmann, P. Agrawal, and A. Torralba. 3d neural scene representations for visuomotor control. In *Conference on Robot Learning*, pages 112–123. PMLR, 2022.

L. Lindemann and D. V. Dimarogonas. Control barrier functions for signal temporal logic tasks. In *Proc. of 57th IEEE Conference on Decision and Control*, 2018. to appear.

H. Liu, Z. Chen, Y. Yuan, X. Mei, X. Liu, D. Mandic, W. Wang, and M. D. Plumbley. Audioldm: Text-to-audio generation with latent diffusion models. *arXiv preprint arXiv:2301.12503*, 2023.

M. Nagumo. Über die lage der integralkurven gewöhnlicher differentialgleichungen. In *Proceedings of the Physico-Mathematical Society of Japan. 3rd Series. 24:551-559*, 1942.

Q. Nguyen and K. Sreenath. Exponential control barrier functions for enforcing high relative-degree safety-critical constraints. In *2016 American Control Conference (ACC)*, pages 322–328. IEEE, 2016.

S. Ozair, Y. Li, A. Razavi, I. Antonoglou, A. Van Den Oord, and O. Vinyals. Vector quantized models for planning. In *International Conference on Machine Learning*, pages 8302–8313. PMLR, 2021.

M. A. Pereira, Z. Wang, I. Exarchos, and E. A. Theodorou. Safe optimal control using stochastic barrier functions and deep forward-backward sdes. In *Conference on Robot Learning*, 2020.

S. Prajna, A. Jadbabaie, and G. J. Pappas. A framework for worst-case and stochastic safety verification using barrier certificates. *IEEE Transactions on Automatic Control*, 52(8):1415–1428, 2007.

M. Preindl. Robust control invariant sets and lyapunov-based mpc for ipm synchronous motor drives. *IEEE Transactions on Industrial Electronics*, 63(6):3925–3933, 2016.

S. V. Rakovic, E. C. Kerrigan, K. I. Kouramas, and D. Q. Mayne. Invariant approximations of the minimal robust positively invariant set. *IEEE Transactions on automatic control*, 50(3):406–410, 2005.

C. Saharia, W. Chan, S. Saxena, L. Li, J. Whang, E. L. Denton, K. Ghasemipour, R. Gontijo Lopes, B. Karagol Ayan, T. Salimans, et al. Photorealistic text-to-image diffusion models with deep language understanding. *Advances in Neural Information Processing Systems*, 35:36479–36494, 2022.

J. Sohl-Dickstein, E. Weiss, N. Maheswaranathan, and S. Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *International Conference on Machine Learning*, pages 2256–2265. PMLR, 2015.

P. Wieland and F. Allgöwer. Constructive safety using control barrier functions. In *Proc. of 7th IFAC Symposium on Nonlinear Control System*, 2007.

R. Wisniewski and C. Sloth. Converse barrier certificate theorem. In *Proc. of 52nd IEEE Conference on Decision and Control*, pages 4713–4718, Florence, Italy, 2013.

W. Xiao and C. Belta. Control barrier functions for systems with high relative degree. In *Proc. of 58th IEEE Conference on Decision and Control*, pages 474–479, Nice, France, 2019.

W. Xiao, T.-H. Wang, R. Hasani, M. Chahine, A. Amini, X. Li, and D. Rus. Barriernet: Differentiable control barrier functions for learning of safe robot control. *IEEE Transactions on Robotics*, 2023a.

W. Xiao, T.-H. Wang, R. Hasani, M. Lechner, Y. Ban, C. Gan, and D. Rus. On the forward invariance of neural odes. In *International conference on machine learning, arXiv preprint arXiv:2210.04763*, 2023b.

H. Zhao, X. Zeng, T. Chen, Z. Liu, and J. Woodcock. Learning safe neural network controllers with barrier certificates. *Form Asp Comp*, 33:437–455, 2021.

S1. Proofs

S1.1. Proof of Theorem 3.2

Proof: Given a continuously differentiable constraint $h(\boldsymbol{x}_t) \geq 0$ ($h(\boldsymbol{x}_0) \geq 0$), by Nagumo's theorem Nagumo (1942), the necessary and sufficient condition for the satisfaction of $h(\boldsymbol{x}_t) \geq 0, \forall t \geq 0$ is

$$\dot{h}(\boldsymbol{x}_t) \geq 0, \ when \ h(\boldsymbol{x}_t) = 0,$$

If $b(\boldsymbol{x}_k^N) \geq 0, k \in \{0, \ldots, H\}$, then the condition (8) is equivalent to

$$\frac{db(\boldsymbol{x}_k^j)}{d\boldsymbol{x}_k^j}\dot{\boldsymbol{x}}_k^j + \alpha(b(\boldsymbol{x}_k^j)) \geq 0,$$

where $\dot{\boldsymbol{x}}_k^j$ is the diffusion time derivative. The last equation is equivalent to

$$\dot{b}(\boldsymbol{x}_k^j) + \alpha(b(\boldsymbol{x}_k^j)) \geq 0,$$

Since α is an extended class $\mathscr{K}$ function, we have that

$$\alpha(b(\boldsymbol{x}_k^j)) \to 0, \ as \ b(\boldsymbol{x}_k^j) \to 0,$$

In other words, we have $\dot{b}(\boldsymbol{x}_k^j) \geq 0$ when $b(\boldsymbol{x}_k^j) = 0$. Since $b(\boldsymbol{x}_k^N) \geq 0, k \in \{0, \ldots, H\}$, then by Nagumo's theorem, we have $b(\boldsymbol{x}_k^j) \geq 0, \forall j \in \{0, \ldots, N-1\}$. Therefore, the diffusion procedure $p_\theta(\boldsymbol{\tau}^{i-1}|\boldsymbol{\tau}^i), i \in \{1, \ldots, N\}$ is finite-time diffusion invariant, and the finite time in diffusion invariance is N.
If, on the other hand, $b(\boldsymbol{x}_k^N) < 0, k \in \{0, \ldots, H\}$, then we can define a Lyapunov function:

$$V(\boldsymbol{x}_k^j) = -b(\boldsymbol{x}_k^j), k \in \{0, \ldots, H\}, j \in \{0, \ldots, N\}, \tag{S1}$$

and $V(\boldsymbol{x}_k^N) > 0$.
Since α is an extended class $\mathscr{K}$ function, replacing $b(\boldsymbol{x}_k^j)$ by $V(\boldsymbol{x}_k^j)$, the condition (8) is equivalent to

$$\frac{dV(\boldsymbol{x}_k^j)}{d\boldsymbol{x}_k^j}\dot{\boldsymbol{x}}_k^j + \alpha(V(\boldsymbol{x}_k^j)) \leq 0,$$

which is equivalent to

$$\dot{V}(\boldsymbol{x}_k^j) + \alpha(V(\boldsymbol{x}_k^j)) \leq 0,$$

Since $\dot{V}(\boldsymbol{x}_k^j) \leq -\alpha(V(\boldsymbol{x}_k^j)) < 0$, we have that $V(\boldsymbol{x}_k^j)$ will be stabilized to 0 by Lyapunov stability theory. In other words, the state $\boldsymbol{x}_k^j$ will be stabilized to the boundary $b(\boldsymbol{x}_k^j) = 0$. Specifically, when α is a linear function, the last equation can be rewritten as

$$\dot{V}(\boldsymbol{x}_k^j) + \varepsilon V(\boldsymbol{x}_k^j)) \leq 0, \tag{S2}$$

where $\varepsilon > 0$. Suppose we have

$$\dot{V}(\boldsymbol{x}_k^j) + \varepsilon V(\boldsymbol{x}_k^j) = 0,$$

the solution to the above equation is

$$V(\boldsymbol{x}_k^j) = V(\boldsymbol{x}_k^N)e^{-\varepsilon(N-j)},$$

Using the comparison lemma Khalil (2002), equation (S2) implies that

$$V(\boldsymbol{x}_k^j) \leq V(\boldsymbol{x}_k^N)e^{-\varepsilon(N-j)}, j \in \{0,\ldots,N\},$$

Therefore,

$$V(\boldsymbol{x}_k^j) \to 0, \textit{ as } j \to 0, \textit{ if } N \textit{ is sufficiently large},$$

and the state $\boldsymbol{x}_k^j$ will be exponentially stabilized to the boundary $b(\boldsymbol{x}_k^j) = 0$. If at diffusion time $j \in \{0,\ldots.N-1\}$, the state $\boldsymbol{x}_k^j$ is close to the boundary, and the probability for the state $\boldsymbol{x}_k^j$ to jump into the set: $\{\boldsymbol{x}_k^j : b(\boldsymbol{x}_k^j) \geq 0\}$ is p (due to the Gaussian transitions in the diffusion process), then the probability for $\boldsymbol{x}_k^0$ to enter the set $\{\boldsymbol{x}_k^j : b(\boldsymbol{x}_k^j) \geq 0\}$ is $1-(1-p)^j$. When j is large enough, then the probability such that $b(\boldsymbol{x}_k^0) \geq 0$ is almost 1. When $b(\boldsymbol{x}_k^l) \geq 0$ at step $l \leq j$, then $b(\boldsymbol{x}_k^r) \geq 0$ for all $r \leq l$ following the Nagumo's theorem (as in the first case of the proof). Therefore, the diffusion procedure $p_\theta(\boldsymbol{\tau}^{i-1}|\boldsymbol{\tau}^i), i \in \{1,\ldots,N\}$ is finite-time diffusion invariant with almost probability 1. ■

S1.2. Proof of Theorem 3.3

Proof: Suppose the weight $w_k(j)$ is chosen such that $w_k(j) = 0$ when $j = 0$, then the condition (10) becomes a hard constraint when $j < 0$. In other words, equation (10) becomes:

$$h(\boldsymbol{u}_k^j|\boldsymbol{x}_k^j) := \frac{db(\boldsymbol{x}_k^j)}{d\boldsymbol{x}_k^j}\boldsymbol{u}_k^j + \alpha(b(\boldsymbol{x}_k^j)) \geq 0, k \in \{0,\ldots,H\}, j \in \{-N_a,\ldots,0\},$$

Then, the proof is simiar to that of the Thm. 3.2, and we have that the diffusion procedure $p_\theta(\boldsymbol{\tau}^{i-1}|\boldsymbol{\tau}^i), i \in \{-N_a,\ldots,N\}$ is finite-time diffusion invariant with almost probability 1. ■

S1.3. Proof of Theorem 3.4

Proof: Since $\gamma_k(N) \leq b(\boldsymbol{x}_k^N)$, we have that $s(\boldsymbol{x}_k^j, \gamma_k(j)) := b(\boldsymbol{x}_k^j) - \gamma_k(j) \geq 0$ when $j = N$.
The condition (12) is equivalent to

$$\frac{\partial s(\boldsymbol{x}_k^j, \gamma_k(j))}{\partial \boldsymbol{x}_k^j}\boldsymbol{u}_k^j + \frac{\partial s(\boldsymbol{x}_k^j, \gamma_k(j))}{\partial j} + \alpha(s(\boldsymbol{x}_k^j, \gamma_k(j))) \geq 0,$$

which can be rewritten as

$$\dot{s}(\boldsymbol{x}_k^j, \gamma_k(j)) + \alpha(s(\boldsymbol{x}_k^j, \gamma_k(j))) \geq 0,$$

Using the Nagumo' theorem presented in the proof of Thm. 3.2, we have that

$$s(\boldsymbol{x}_k^j, \gamma_k(j)) \geq 0, \forall j \in \{0,\ldots,N\}$$

since $s(\boldsymbol{x}_k^N, \gamma_k(N)) \geq 0$.
As $\gamma_k(0) = 0$ and $s(\boldsymbol{x}_k^j, \gamma_k(j)) := b(\boldsymbol{x}_k^j) - \gamma_k(j)$, we have that $b(\boldsymbol{x}_k^0) \geq 0, \forall k \in \{0,\ldots,H\}$. Therefore, the diffusion procedure $p_\theta(\boldsymbol{\tau}^{i-1}|\boldsymbol{\tau}^i), i \in \{0,\ldots,N\}$ is finite-time diffusion invariant, and the finite time in diffusion invariance is 0. ■

S2. Experiment Details

S2.1. Safe Planning in Maze

In this experiment, we aim to impose trajectory constraints on the planning path of a maze. The training data is publicly available from Janner et al. (2022), in which initial positions and destinations in maze are randomly generated. The diffusion model is conditioned on the initial positions and destinations.

Specifications. The simple safety specification for the planning trajectory is defined as an super-ellipse-shape obstacle:

$$\left(\frac{x-x_0}{a}\right)^2+\left(\frac{y-y_0}{b}\right)^2 \geq 1, \tag{S3}$$

where $(x,y) \in \mathbb{R}^2$ is the state on the planning trajectory, $(x_0,y_0) \in \mathbb{R}^2$ is the location of the obstacle. $a>0, b>0$. Since the state (x,y) is normalized in diffusion models, we also need to normalize the above constraint accordingly. In other words, we normalize x_0, a and y_0, b according to the normalization of (x,y) along the x-axis and $y-$axis, respectively.

The complex safety specification for the planning trajectory is defined as an ellipse-shape obstacle:

$$\left(\frac{x-x_0}{a}\right)^4+\left(\frac{y-y_0}{b}\right)^4 \geq 1, \tag{S4}$$

We also normalize the above constraint as in the simple case. In this case, it is non-trivial to truncate the planning trajectory to satisfy the constraint. When we have much more complex specifications, it is too hard for the truncation method to work.

Model setup, training and testing. The diffusion model structure is the same as the open source one (Maze2D-large-v1) provided in Janner et al. (2022). We set the planning horizon as 384, the diffusion steps as 256 with an additional $N_a = 50$ diffusion steps for the proposed methods. The learning rate is $2e^{-4}$ with $2e^6$ training steps. The training of the model takes about 10 hours on a Nvidia RTX-3090 GPU. More parameters are provided in the attached code: "safediffuser/config/maze2d.py". The switch of different (proposed) methods in testing can be modified in "safediffuser/diffuser/models/diffusion.py" through "GaussianDiffusion.p_sample()" function.

In Fig. S1, we present a diffusion procedure using the diffuser, in which case the generated trajectory can easily violate safety constraints. Using the proposed robust-safe diffuser, the generated trajectory can guarantee safety, but some points on the trajectory may get stuck in local traps, as shown in S2. Using the proposed relaxed-safe diffuser and time-varying-safe diffuser, the local trap problem could be addressed.

S2.2. Safe Planning for Robot Locomotion

For robot locomotion (in MuJoCo), we wish the robot to avoid collisions with obstacles, such as the roof. In this case, since there is no local trap problem, we only consider robust-safe diffuser (RoS-diffuser). Others work similarly. The training data set is publicly available from Janner et al. (2022).

Specifications. The simple safety specification for both the Walker2D and Hopper is collision avoidance with the roof. In other words, the height of the robot head $z \in \mathbb{R}$ should satisfy the following constraint:

$$z \leq h_r, \tag{S5}$$

where $h_r > 0$ is the height of the roof. We also need to normalize h_r according to the normalization of the state z in the diffusion model.

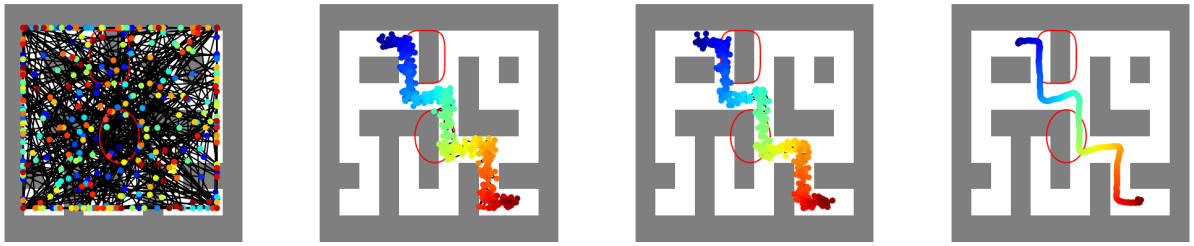

Fig. S1: Maze planning (blue to red) denoising diffusion procedure with diffuser (Left to right: diffusion time steps 256, 4, 3, 0, respectively). Red ellipse and superellise (outside) denote safe specifications. Both specifications are violated with the trajectory from diffuser.

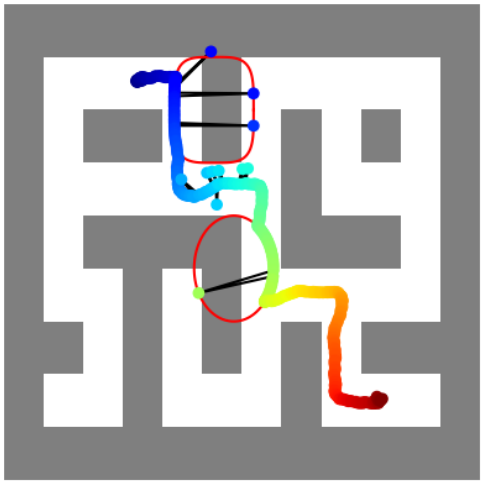

Fig. S2: Maze planning (blue to red) denoising diffusion procedure with robust-safe diffuser at diffusion time step 0. Red ellipse and superellise (outside) denote safe specifications. Although with safety guarantees, some trajectory points may get stuck in local traps.

The complex safety specification for both the Walker2D and Hopper is a speed-dependent collision avoidance constraint:

$$z + \varphi v_z \leq h_r, \tag{S6}$$

where $\varphi > 0$, $v_z \in \mathbb{R}$ is the speed of the robot head along the z-axis. The speed-dependent safety constraint is more robust for the robot to avoid collision with the roof since when the robot jumps faster, we need to ensure a larger safe distance with respect to the roof in order to account for all kinds of uncertainties or perturbations. In this case, the simple truncation method is hard to work since it is not clear how to truncate both z and v_z at the same time.

Model setup, training and testing. The diffusion model structures are the same as the open source ones (Walker2D-Medium-Expert-v2 and Hopper-Medium-Expert-v2) provided in Janner et al. (2022). We set the planning horizon as 600, the diffusion steps as 20. The learning rate is $2e^{-4}$ with $2e^{6}$ training steps. The training of the model takes about 16 hours on a Nvidia RTX-3090 GPU. More parameters are provided in the attached code: "safediffuser/config/locomotion.py". The switch of different methods in testing can be modified

in "safediffuser/diffuser/models/diffusion.py" through "GaussianDiffusion.p_sample()" function.

S2.3. Safe Planning for Manipulation

For manipulation (in Pybullet), the diffusion models generate joint trajectories (as controls) for the robot, which are conditioned on the locations of the objects to grasp and place. The training data set is publicly available from Janner et al. (2022). Specifications are joint limitations to avoid collision in joint space.

Specifications. The simple safety specification for the robot is in the joint space, and we are trying to limit the joint angles of the robot within allowed ranges:

$$x_{min} \leq x \leq x_{max}, \tag{S7}$$

where $x \in \mathbb{R}^7$ is the state of 7 joint angles, $x_{min} \in \mathbb{R}^7$ and $x_{max} \in \mathbb{R}^7$ denotes the minimum and maximum joint limits. We need to normalize the limits according to how the state x is normalized in the diffusion model.

The complex safety specifications are speed-dependent joint constraints:

$$x_{min} \leq x + \varphi v \leq x_{max}, \tag{S8}$$

where $\varphi > 0$, $v \in \mathbb{R}^7$ is the joint speed corresponding to the joint angle x. In this example, since the diffusion model does not directly predict v, we evaluate v using $x(k)$ and $x(k+1)$ along the planning horizon. The joints limits are also normalized as in the simple specification case.

Model setup, training and testing. The diffusion model structure is the same as the open source one provided in Janner et al. (2022), and we use their pre-trained models to evaluate our methods when comparing with other approaches.